

Algorithmic Specifications

Dynamic Multi-Primitive Cryptographic Hopping Protocol (DMP-CHP)

March 25, 2026

1 Core Algorithms

1.1 Stateless Schedule Derivation (Resilient Hopping)

To handle packet loss and out-of-order delivery (e.g., in UDP/QUIC streams), the hopping state depends on a monotonic sequence number rather than strictly the previous internal state.

Algorithm 1 Get_Hop_Parameters

Require: Master secret K_{session} , packet sequence number SeqID

Require: Library size N_{algo} , mode Mode

Ensure: Algorithm Index idx , Packet Key k_{pkt}

- 1: Salt $\leftarrow$ Mode.salt
 - 2: Seed $\leftarrow$ HMAC(K_{session} , "HOP" || SeqID || Salt) $\triangleright$ Use the first 32 bits of Seed in network byte order for index derivation
 - 3: $idx \leftarrow$ Trunc32(Seed[0:4] (network byte order)) mod N_{algo}
 - 4: $k_{pkt} \leftarrow$ HKDF(Seed, info = "KEYGEN")
- return** (idx, k_{pkt})
-

1.2 Holographic Entropy Dispersion (Encryption)

1.3 Active Decoy Injection

1.4 Neuro-Cognitive Adaptation

Algorithm 2 Holographic_Encrypt

Require: Session secret K_{session} **Require:** Plaintext payload P , threshold k , total shares n , base sequence BaseSeqID**Require:** Mode Mode, max orthogonality retries R_{max} **Require:** Orthogonality library Λ (array of algorithms), library size N_{algo} **Ensure:** Composite Ciphertext C_{bundle}

```
1:  $Shares \leftarrow \text{Shamir\_Split}(P, k, n)$  ▷ Generate  $n$  shares
2:  $C_{\text{bundle}} \leftarrow []$ 
3:  $\text{prev\_class} \leftarrow \perp$  ▷ No previous hard-problem class for first share
4: for  $i \leftarrow 1$  to  $n$  do
5:    $s_i \leftarrow Shares[i]$ 
6:    $\text{SeqID} \leftarrow \text{BaseSeqID} + i$  ▷ Transmitted / authenticated monotonic sequence identifier
7:    $\text{SelCtr} \leftarrow \text{SeqID}$  ▷ Selection counter used for enforcing orthogonality
8:    $(idx, key) \leftarrow \text{Get\_Hop\_Parameters}(K_{\text{session}}, \text{SelCtr}, N_{\text{algo}}, \text{Mode})$ 
9:    $Algo \leftarrow \Lambda[idx]$  ▷ Select algorithm for this share
   ▷ Ensure Orthogonality: retry until hard-problem class differs from previous
10:   $\text{retry} \leftarrow 0$ 
11:  while  $Algo.\text{hard\_problem\_class} = \text{prev\_class}$  do
12:     $\text{SelCtr} \leftarrow \text{SelCtr} + 1$  ▷ Deterministic bump to escape same-class selection
13:     $(idx, key) \leftarrow \text{Get\_Hop\_Parameters}(K_{\text{session}}, \text{SelCtr}, N_{\text{algo}}, \text{Mode})$ 
14:     $Algo \leftarrow \Lambda[idx]$ 
15:     $\text{retry} \leftarrow \text{retry} + 1$ 
16:    if  $\text{retry} > R_{\text{max}}$  then
17:      break ▷ Fallback after  $R_{\text{max}}$  attempts; orthogonality may be relaxed
18:    end if
19:  end while
20:   $c_i \leftarrow Algo.\text{Encrypt}(key, s_i, \text{AD} = \text{Header}(\text{SeqID}))$  ▷ Use AEAD-style Encrypt(key, plaintext, AD) to bind SeqID
21:   $C_{\text{bundle}}.\text{append}(\{c_i, \text{Header}(\text{SeqID})\})$  ▷ Header contains authenticated SeqID bound as AD
22:   $\text{prev\_class} \leftarrow Algo.\text{hard\_problem\_class}$ 
23: end for
  return  $C_{\text{bundle}}$ 
```

Algorithm 3 Generate_Decoys

Require: Legitimate traffic distribution $\mathcal{D}_{\text{traffic}}$, current schedule**Require:** Session secret K_{session} **Require:** Administrative/control key K_{admin} (or a control key derivable from K_{session})**Require:** Orthogonality library Λ (array of algorithms), library size N_{algo} , mode Mode**Require:** Ghost sequence counter GhostSeqID (monotonic counter stored in state; initialize to 0 if undefined)**Ensure:** Decoy Packet P_{decoy}

```
1:  $\text{TargetLen} \leftarrow \text{Sample}(\mathcal{D}_{\text{traffic}}.\text{length})$ 
2:  $\text{TargetInterarrival} \leftarrow \text{Sample}(\mathcal{D}_{\text{traffic}}.\text{timing})$ 
3:  $\text{Noise} \leftarrow \text{TRNG}(\text{TargetLen})$ 
4:  $\text{GhostSeqID} \leftarrow \text{GhostSeqID} + 1$  ▷ Monotonic counter reserved for decoys (stateful)
5:  $(idx, key) \leftarrow \text{Get\_Hop\_Parameters}(K_{\text{session}}, \text{GhostSeqID}, N_{\text{algo}}, \text{Mode})$ 
6:  $Algo \leftarrow \Lambda[idx]$ 
7:  $P_{\text{decoy}} \leftarrow Algo.\text{Encrypt}(key, \text{Noise})$ 
8:  $P_{\text{decoy}}.\text{Header}.\text{Flag} \leftarrow \text{Encrypted}(\text{"DECOY"}, K_{\text{admin}})$ 
  return  $P_{\text{decoy}}$ 
```

Algorithm 4 Update_Threat_State

Require: Current State $State_t$, Network Metrics M_t , Global Model θ_{global}

Ensure: New Mode $Mode_{t+1}$

```
1:  $Features \leftarrow \text{Extract\_Features}(M_t)$ 
2:  $\text{ThreatScore} \leftarrow \text{NeuralNet}_{\theta_{global}}(Features)$ 
3:  $\text{LocalEntropy} \leftarrow \text{Measure\_Jitter}()$ 
4: if  $\text{ThreatScore} > T_{\text{paranoid}}$  then
5:    $Mode_{t+1} \leftarrow \text{"NANO\_HOPPING"}$ 
6:    $\text{Inject\_Chaff}(Rate = HIGH)$ 
7: else if  $\text{ThreatScore} > T_{\text{alert}}$  then
8:    $Mode_{t+1} \leftarrow \text{"MICRO\_HOPPING"}$ 
9: else
10:   $Mode_{t+1} \leftarrow \text{"MACRO\_HOPPING"}$ 
11: end if
12:  $\text{Update\_Locally\_Learned\_Weights}()$   $\triangleright$  Federated Learning Step return  $Mode_{t+1}$ 
```
